

Homework 05

Rahaf Alabdullah (rahafaa2)

Question 1: Estimate the bias and variance of KNN

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsRegressor

#part a
rng = np.random.default_rng(43251)
n = 400
p = 3
k_values = np.arange(1, 30, 2)
x0 = np.array([[0.5, 0.7, 1]])
f0_true = 0.5*x0[0, 0] + np.sin(x0[0, 1]) - 0.3*x0[0, 2]**2
x = rng.normal(loc=0.0, scale=1.0, size=(n, p))
mu_train = 0.5*x[:, 0] + np.sin(x[:, 1]) - 0.3*x[:, 2]**2
epsilon = rng.normal(loc=0.0, scale=0.1, size=n)
y_train = mu_train + epsilon
results_fitted_k = np.empty(len(k_values))
for l, k in enumerate(k_values):
    knn_fit = KNeighborsRegressor(n_neighbors=k, weights="uniform")
    knn_fit.fit(x, y_train)
    pred_value_per_k = knn_fit.predict(x0)
    results_fitted_k[l] = pred_value_per_k[0]

print(pd.DataFrame({"k": k_values, "prediction": results_fitted_k, "truth": f0_true}))

#part b
repetitions = 200
results_b = np.empty((repetitions, len(k_values)))
for i in range(repetitions):
    x_train_b = rng.normal(loc=0.0, scale=1.0, size=(n, p))
    epsilon_b = rng.normal(loc=0.0, scale=0.1, size=n)
    mu_train_b = 0.5*x_train_b[:, 0] + np.sin(x_train_b[:, 1]) - 0.3*x_train_b[:, 2]**2
    y_train_b = mu_train_b + epsilon_b
    for l, k in enumerate(k_values):
        knn_fit = KNeighborsRegressor(n_neighbors=k, weights="uniform")
        knn_fit.fit(x_train_b, y_train_b)
        pred_b = knn_fit.predict(x0)
        results_b[i, l] = pred_b[0]
bias = np.empty(len(k_values))
variance = np.empty(len(k_values))
MSE = np.empty(len(k_values))
for l, k in enumerate(k_values):
    k_column = results_b[:, l]
    bias[l] = np.mean(k_column) - f0_true
    variance[l] = np.var(k_column)
    MSE[l] = np.mean((k_column - f0_true)**2)

print("max |MSE - (bias^2 + variance)|:", np.max(np.abs(MSE - (bias**2 + variance))))

plt.figure(figsize=(8, 6))
plt.plot(k_values, bias**2, marker="o", label="Squared Bias")
plt.plot(k_values, variance, marker="o", label="Variance")
plt.plot(k_values, MSE, marker="o", label="MSE")
plt.xlabel("k")
plt.ylabel("Value")
plt.title("Bias-Variance Trade-off for KNN at x0")
plt.legend()
plt.tight_layout()
plt.show()
```

(a) KNN prediction at x_0 for each k

$$f(x_0) = 0.5942.$$

k	Prediction	k	Prediction	k	Prediction
1	0.3696	11	0.3428	21	0.3995
3	0.4356	13	0.4083	23	0.3591
5	0.4821	15	0.3803	25	0.2977
7	0.4505	17	0.4149	27	0.3370
9	0.3712	19	0.3898	29	0.3180

(b) Bias, variance, and MSE

The numerical check confirms $\text{MSE} = \text{squared bias} + \text{variance}$: the largest observed discrepancy across all k was 1.7×10^{-17} , i.e. floating-point noise.

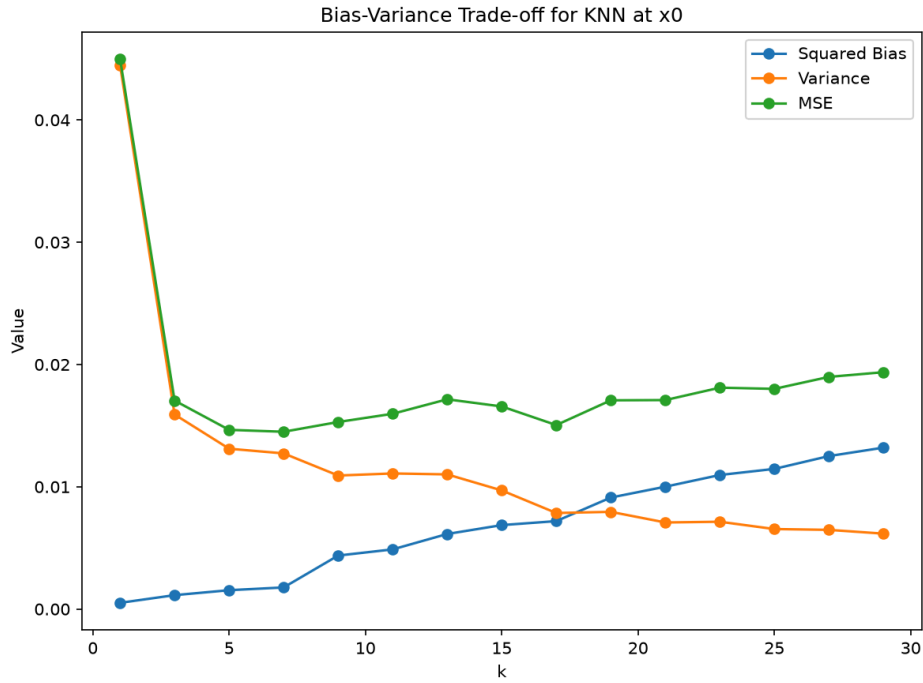


Figure 1: Squared bias, variance, and MSE vs. k

As k increases, squared bias steadily increases while variance steadily decreases — averaging over more, and increasingly distant, neighbors smooths out the fitted mean (raising bias) while reducing sensitivity to the noise in any one training set (lowering variance). MSE is minimized at a small-to-moderate k , where the two effects balance. This mirrors the bias-variance trade-off in ridge and lasso, except that the trade-off there is controlled by the penalty strength λ rather than by neighborhood size: small λ (like small k) gives low bias and high variance, while large λ (like large k) gives high bias and low variance.

Question 2: Compare KNN with lasso

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Lasso
rng = np.random.default_rng(43252)
n = 400
p = 30
lambda_grid = np.logspace(0, -4, 41)
x0 = np.array([[0.5, 0.7, 1]])
f0_true = 0.5*x0[0, 0] + np.sin(x0[0, 1]) - 0.3*x0[0, 2]**2
results = pd.DataFrame({
    "Setting": ["Independent", "Correlated"],
    "KNN_MSE": np.nan, "Lasso_MSE": np.nan,
    "Lambda": np.nan, "Nonzero": 0
})
for i in range(2):
    Sigma = np.eye(p)
    if i == 1:
        correlated = np.full((p, p), 0.8)
        np.fill_diagonal(correlated, 1.0)
        Sigma = correlated

    mean = np.zeros(p)
    x = rng.multivariate_normal(mean=mean, cov=Sigma, size=n + 1000)
    y = (0.5 * x[:, 0] + np.sin(x[:, 1]) - 0.3 * x[:, 2]**2
        + np.random.normal(scale=0.1, size=n + 1000))
    train_X, test_X = x[:n], x[n : n+1000]
    train_y, test_y = y[:n], y[n : n+1000]

    knn_fit = KNeighborsRegressor(n_neighbors=5, weights="uniform",
                                algorithm="brute")
    knn_fit.fit(train_X, train_y)

    results.loc[i, "KNN_MSE"] = np.mean((test_y - knn_fit.predict(test_X))**2)

    fold = rng.permutation(np.repeat(np.arange(10), n // 10))
    CV_mse = np.zeros((10, len(lambda_grid)))
    for m in range(10):
        X_mean = train_X[fold != m, :].mean(axis=0)
        X_sd = train_X[fold != m, :].std(axis=0, ddof=0)
        X_fit = (train_X[fold != m, :] - X_mean) / X_sd
        X_valid = (train_X[fold == m, :] - X_mean) / X_sd
        y_mean = train_y[fold != m].mean()
        y_tilda = train_y[fold != m] - y_mean

        for lam in range(len(lambda_grid)):
            lasso_fit = Lasso(alpha=lambda_grid[lam], fit_intercept=False,
                              tol=1e-8, max_iter=100000)
            lasso_fit.fit(X_fit, y_tilda)
            y_hat = lasso_fit.predict(X_valid) + y_mean
            CV_mse[m, lam] = np.mean((train_y[fold == m] - y_hat)**2)
        selected_penalty = lambda_grid[np.argmin(CV_mse.mean(axis=0))]

    #refit on all training observations
    X_mean = train_X.mean(axis=0)
    X_sd = train_X.std(axis=0, ddof=0)
    X_fit = (train_X - X_mean) / X_sd
    test_X_fit = (test_X - X_mean) / X_sd
    y_mean = train_y.mean()
    lasso_fit = Lasso(alpha=selected_penalty, fit_intercept=False,
                      tol=1e-8, max_iter=100000)
    lasso_fit.fit(X_fit, train_y - y_mean)
    test_pred = lasso_fit.predict(test_X_fit) + y_mean
    results.loc[i, "Lasso_MSE"] = np.mean((test_y - test_pred)**2)
    results.loc[i, "Lambda"] = selected_penalty
    results.loc[i, "Nonzero"] = np.count_nonzero(lasso_fit.coef_)
print(results.round(4).to_string(index=False))
```

(a) Test MSE, selected penalty, and nonzero slopes

Setting	KNN MSE	Lasso MSE	Selected λ	Nonzero slopes
Independent	0.5807	0.2621	0.0501	5
Correlated	0.1659	0.2445	0.0200	7

(b) KNN does better in the correlated setting because Euclidean distance weighs all 30 covariates equally. When the irrelevant covariates are independent, they add pure noise to the distance and corrupt the neighborhoods; when they are correlated with X_1, X_2, X_3 , points close in those three are also close along the rest, so the noise is less disruptive. Lasso is not affected this way, since it zeroes out irrelevant coefficients directly — which is why it beats KNN in the independent setting.

(c) KNN has no fixed functional form, so it can approximate $\sin(X_2)$ and X_3^2 , which lasso cannot represent at all. But KNN's accuracy still depends on finding informative neighbors, as in part (b). One dataset per setting only shows what happened here, not a general rule: it does not show that correlation always helps KNN, or that either method is always better.

Question 3: Observed dimension and latent dimension

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsRegressor

#part a
rng = np.random.default_rng(43254)
p = 100
m_values = [3, 30]
k_values = np.arange(2, 83, 8)
test_mse=np.empty((len(k_values), len(m_values)))
for s, m in enumerate(m_values):
    A = rng.uniform(-2, 2, size=(p, m))
    Z = rng.normal(size=(400, m))
    eta = rng.normal(size=(400, p))
    X = Z @ A.T + eta
    y = Z[:, 0] + Z[:, 1] + rng.normal(size=400)
    for i, k in enumerate(k_values):
        knn_fit = KNeighborsRegressor(n_neighbors=int(k), weights="uniform",
                                     algorithm="brute")
        knn_fit.fit(X[:200], y[:200])
        test_mse[i, s] = np.mean((y[200:] - knn_fit.predict(X[200:]))**2)
print(pd.DataFrame({"k": k_values, "m=3": test_mse[:, 0],
                   "m=30": test_mse[:, 1]}))

#part b
repetitions= 50
test_mse_b=np.empty((repetitions, len(k_values), len(m_values)))
for j in range(repetitions):
    for s, m in enumerate(m_values):
        A = rng.uniform(-2, 2, size=(p, m))
        Z = rng.normal(size=(400, m))
        eta = rng.normal(size=(400, p))
        X = Z @ A.T + eta
        y = Z[:, 0] + Z[:, 1] + rng.normal(size=400)
        for t, k in enumerate(k_values):
            knn_fit = KNeighborsRegressor(n_neighbors=int(k), weights="uniform",
                                         algorithm="brute")
            knn_fit.fit(X[:200], y[:200])
            test_mse_b[j, t, s] = np.mean((y[200:] - knn_fit.predict(X[200:]))**2)
mean_mse = test_mse_b.mean(axis=0)
print(pd.DataFrame({"k": k_values, "m=3": mean_mse[:, 0],
                   "m=30": mean_mse[:, 1]}))

plt.figure(figsize=(7, 4.5))
plt.plot(k_values, mean_mse[:, 0], "o-", color="#009E73", label="m = 3")
plt.plot(k_values, mean_mse[:, 1], "^-", color="#56B4E9", label="m = 30")
plt.xlabel("Number of neighbors k")
plt.ylabel("Average test MSE")
plt.legend()
plt.show()
```

(a) Test MSE for one dataset

k	$m = 3$	$m = 30$	k	$m = 3$	$m = 30$
2	1.7342	3.4247	42	1.5286	2.7057
10	1.3484	2.8549	50	1.5793	2.7565
18	1.4094	2.7042	58	1.6406	2.8065
26	1.4515	2.6896	66	1.6900	2.8333
34	1.4928	2.7084	74	1.7528	2.8689
			82	1.8116	2.9115

(b) Average test MSE over 50 repetitions

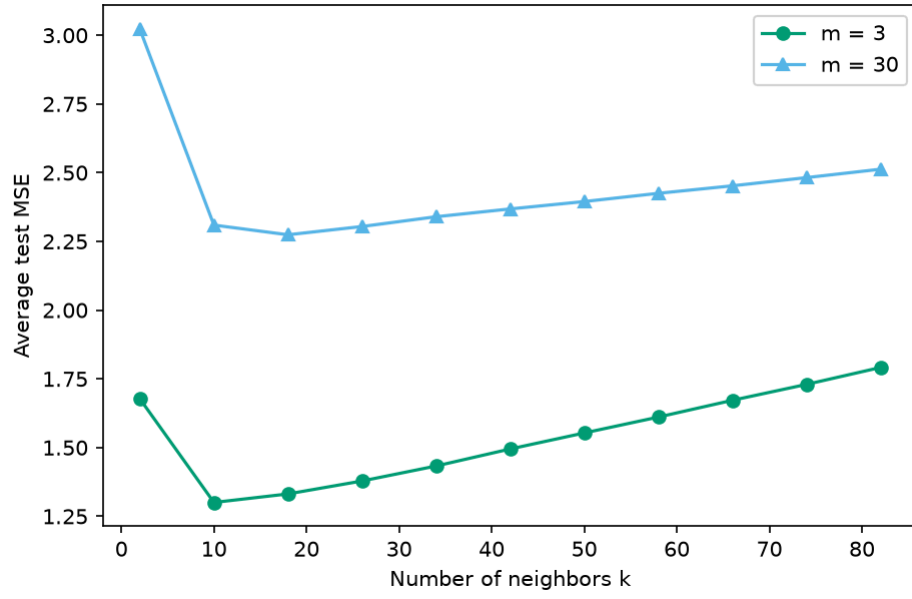


Figure 2: Average test MSE vs. k , for $m=3$ and $m=30$

(c) With $m = 3$, the noise η still pushes points off the exact span of $\mathbf{A}$, so nearby $\mathbf{Z}$'s are not always nearby in $\mathbf{X}$. With $m = 30$, distance also picks up 28 extra latent directions that don't affect Y , adding noise to the neighborhoods and giving $m = 30$ a higher test MSE.

Question 4: Classify handwritten digits

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

zip_train = np.loadtxt("data/knn/zip-train.csv.gz", delimiter=",")
zip_test = np.loadtxt("data/knn/zip-test.csv.gz", delimiter=",")

#part a
X_train = zip_train[:, 1:]
y_train = zip_train[:, 0].astype(int)
X_test = zip_test[:, 1:]
y_test = zip_test[:, 0].astype(int)

def euclidean_distance(x0, X):
    return np.sqrt(np.sum((X - x0)**2, axis=1))

def manhattan_distance(x0, X):
    return np.sum(np.abs(X - x0), axis=1)

def knn_function(X_train, y_train, X_test, distance):
    y_hat = np.zeros(X_test.shape[0], dtype=int)
    for i in range(X_test.shape[0]):
        d = distance(X_test[i], X_train)
        neighbors = np.argsort(d, kind="stable")[:5]
        votes = np.bincount(y_train[neighbors], minlength=10)
        y_hat[i] = np.argmax(votes)
    return y_hat
y_hat = np.column_stack((knn_function(X_train, y_train, X_test, euclidean_distance),
                          knn_function(X_train, y_train, X_test, manhattan_distance)))

#part b
for j in range(2):
    print(["Euclidean", "Manhattan"][j],
          "test error:", np.mean(y_hat[:, j] != y_test))
    print(pd.crosstab(pd.Series(y_hat[:, j], name="Predicted"),
                     pd.Series(y_test, name="True"))
          .reindex(index=range(10), columns=range(10), fill_value=0))

#part c
incorrect = np.flatnonzero(y_hat[:, 0] != y_test)[:3]
plt.figure(figsize=(9, 3))
for i in range(3):
    idx = incorrect[i]
    plt.subplot(1, 3, i + 1)
    plt.imshow(X_test[idx].reshape(16, 16), cmap="gray_r")
    plt.title(f"True: {y_test[idx]}, Pred: {y_hat[idx, 0]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

(a) The 5NN classifier was implemented from scratch: distances are computed directly with NumPy (no `sklearn/scipy` distance or KNN calls), the 5 nearest neighbors are found with a stable sort (ties broken toward the smaller training row number), and the predicted label is the most frequent digit among them via `np.bincount` + `np.argmax` (ties broken toward the smallest digit, since `argmax` returns the first maximal index and `bincount`'s index is the digit value).

(b) Test classification error

Metric	Test error
Euclidean	0.122
Manhattan	0.138

Confusion matrix — Euclidean distance (rows: predicted, columns: true)

Pred \ True	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0

Pred \ True	0	1	2	3	4	5	6	7	8	9
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

Confusion matrix — Manhattan distance (rows: predicted, columns: true)

Pred \ True	0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

For both metrics, digit **9** is most often confused with **4**, and digit **0** is occasionally confused with **2** and **8** — pairs that share a similar rounded stroke shape.

(c)

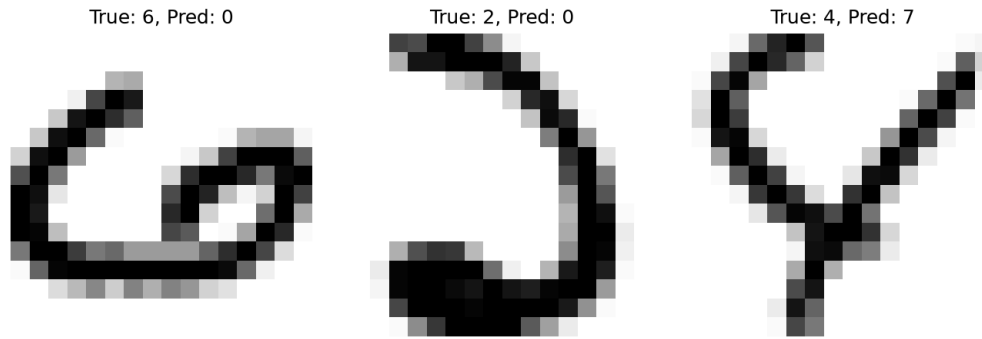


Figure 3: Three misclassified test digits (Euclidean)

Given these shapes, it makes sense that a pixel-distance rule would stumble here, even though a person could still read the digit: human recognition draws on cues about handwriting structure that this distance measure never explicitly captures. Looking at eight further images, both metrics mistake these images but land on different wrong digits, so two methods can rack up similar total error counts while disagreeing on which cases they get wrong. Euclidean distance edges out Manhattan on this particular split, but that alone doesn't prove it is the better metric in general.

Question 5: Create a plot-formatting skill

(a) SKILL.md, stored at `.github/skills/plot-formatting/SKILL.md`:

```
---
name: plot-formatting
description: "Use only when explicitly asked to use plot-formatting to improve the formatting of a STAT 432 homework plot, in R or Python."
---

# Plot Formatting

When I explicitly ask you to use this skill, revise the formatting of a plot
I already produced for a STAT 432 homework, in R or Python. Do not change
what the plot shows, only how it looks.

## Formatting rules

- Margins: Auto-fit the layout so titles, labels, and ticks are never
  clipped or overlapping, with no excess whitespace around the plot
- Title: centered, bold
- Axes: remove the top and right border lines, keeping only the left and
  bottom axis lines
- Labels: Parameter Name (units)
- Colors: foreground #000000, a colorblind-friendly qualitative palette,
  background #FFFFFF
- Sizes: figure dimensions (6.5, 4.0) inches, resolution 100-150 DPI,
  title font 14pt, axis label font 11pt
```

(b) The skill was installed in this repository's skills folder and explicitly invoked on the Question 1 bias-variance plot.

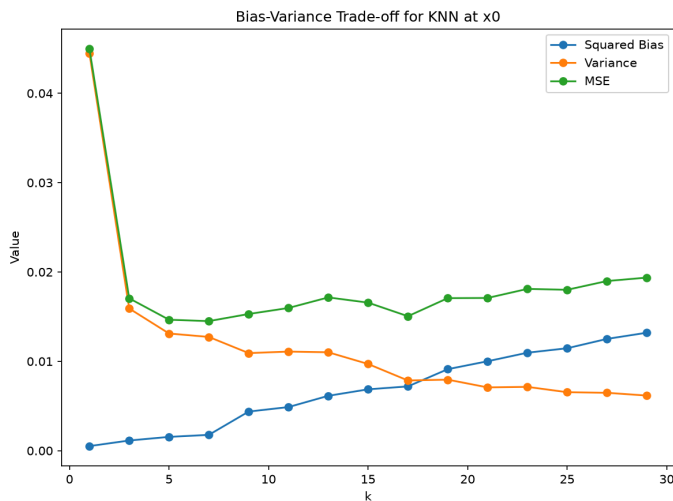


Figure 1: Original

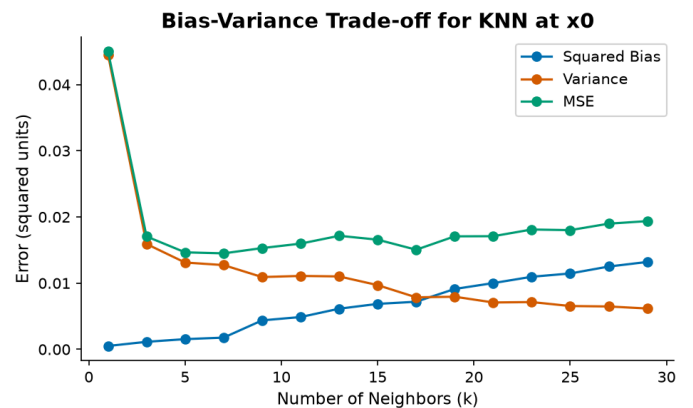


Figure 2: Revised, using plot-formatting

(Your reflection: do you like the changes? Note anything you'd revise about the skill or keep as is.)